



Notes d'oral

Scraper books.toscrape.com

Victorien Alleg — Session 3
Eugenia School — Data Pipelines for IA
Mai 2026

1 Présentation du projet

1 050	1 000	< 11s	100%
requêtes HTTP	livres scrapés	durée totale	taux de succès

Ce que je dois dire :

- Le projet consiste à scraper l'intégralité de books.toscrape.com — 1 000 livres sur 50 pages.
- L'objectif était de le faire en moins de 11 secondes, de manière fiable et automatisée via Docker.
- Le résultat : 1 050 requêtes HTTP traitées en ~10 secondes, 0 erreur, CSV propre à chaque run.

■ *Commencer par les chiffres donne une bonne impression — montre que tu sais ce que tu as produit.*

2 Architecture — 2 phases parallèles

Phase 1 — Catalogue (50 requêtes)

- Les URLs sont prévisibles : page-1.html, page-2.html... page-50.html.
- Au lieu de paginer séquentiellement, on génère les 50 URLs et on les télécharge toutes en même temps.
- Résultat : 8.6s → 1.6s. Gain $\times 5$ sur cette phase.

Phase 2 — Fiches livres (1 000 requêtes)

- Toutes les URLs collectées en Phase 1 sont soumises d'un coup au ThreadPoolExecutor (50 workers).
- Les threads tournent en parallèle — au lieu d'attendre 1 réponse à la fois, on en traite 50 simultanément.
- Résultat : ~8.6s pour 1 000 requêtes.

Connection pooling

- Sans session HTTP : chaque requête ouvre une nouvelle connexion TCP → +100ms par requête.
- Avec requests.Session + HTTPAdapter : les connexions sont réutilisées (HTTP keep-alive).
- Sur 1 000 requêtes, c'est plusieurs dizaines de secondes économisées.

■ Si on te demande « pourquoi 2 phases et pas 1 ? » : Phase 1 collecte les URLs, Phase 2 les utilise. On ne peut pas paralléliser sans connaître les cibles.

3 Concepts clés — questions possibles

■ C'est quoi un ThreadPoolExecutor ?

Un pool de threads Python. On lui soumet des tâches (fonctions à exécuter), il les distribue sur N threads qui tournent en parallèle. Ici : 50 threads qui chacun scrape une fiche livre simultanément.

■ Pourquoi un Lock sur le compteur de requêtes ?

Sans Lock, deux threads peuvent lire la même valeur (ex: 42) en même temps, l'incrémenter chacun, et écrire 43 deux fois → valeur fausse. Le Lock force un accès exclusif : un thread à la fois modifie le compteur.

■ C'est quoi le backoff exponentiel ?

En cas d'erreur réseau, on attend avant de réessayer. Mais au lieu d'attendre toujours la même durée, on double à chaque tentative : 2s → 4s → abandon. Ça évite de surcharger un serveur déjà en difficulté.

■ Pourquoi Docker ?

Pour que le code tourne identiquement sur n'importe quelle machine. L'image contient Python, les dépendances, et cron qui relance automatiquement le scraper toutes les minutes.

■ Comment les logs apparaissent dans docker logs ?

cron exécute scraper.py mais son stdout n'est pas redirigé vers le container. Solution : entrypoint.sh lance cron puis fait tail -f sur le fichier de log, ce qui redirige le fichier vers stdout du container → visible dans docker logs.

■ Pourquoi remettre le CSV à zéro à chaque run ?

Pour avoir toujours une photo fraîche du site. Sans réinitialisation, le fichier grossit indéfiniment (4 000 lignes après 4 runs). `init_csv()` ouvre le fichier en mode 'w' (écrase) au démarrage du pipeline.

4 Évolution des performances

Version	Méthode	Durée
Session 1 — naïve	Séquentiel, 1 req à la fois	~3 minutes
Session 3 — 8 workers	Phase 1 séq. + Phase 2 //	~42 secondes
Session 3 — 20 workers	Phase 1 séq. + Phase 2 //	~19 secondes
Session 3 — 50 workers (version finale)	Phases 1 ET 2 en parallèle	< 11 secondes ✓

Le chiffre à retenir :

- Gain total : $\times 18$ entre la version naïve (3 min) et la version finale (10s).
- Le plus grand gain vient de la parallélisation de la Phase 1 : $\times 5$ sur cette seule phase.

■ Si on te demande comment tu as mesuré : `time python3 scraper.py` + les logs affichent les durées de chaque phase.

5 Problèmes rencontrés & solutions

■ Docker logs vides

cron ne redirige pas stdout vers le container. Fix : entrypt.sh fait tail -f sur le fichier de log.

■ CSV qui s'accumule (4 000 lignes)

init_csv était appelé en Phase 2, pas au démarrage. Fix : déplacement de init_csv en tout début de run(), mode 'w'.

■ Phase 1 lente (8.6s séquentiel)

Pagination séquentielle = attendre chaque réponse avant la suivante. Fix : URLs prévisibles → téléchargement en parallèle → 1.6s.

■ Race condition sur le compteur

Deux threads lisaient la même valeur simultanément. Fix : threading.Lock() protège l'incrément.

6 Conclusion & liens utiles

Ce que j'ai produit :

- scraper.py : pipeline 2 phases, 50 workers, connection pooling, logs structurés, arrêt propre
- Docker : container avec cron toutes les minutes, logs visibles via docker logs
- GitHub Actions : scraping quotidien automatique, notification email si échec
- CSV propre : 1 000 livres (titre, prix, note, catégorie, date) réinitialisé à chaque run

Améliorations possibles :

- Export en base de données (PostgreSQL) pour historiser les données
- Dashboard de monitoring temps réel (logs JSON + Grafana)
- Notifications Slack/Discord en cas d'échec

Liens :

GitHub github.com/valleg12/books-scraper-session3

Site scrapé books.toscrape.com

■ Terminer par 'Le projet est fonctionnel, dans le repo, et tourne en production via Docker' — montre que c'est livré, pas juste un exercice.